Day 3：投资问效案例 – 多智能体协作与Plan–Execute–Judge循环

编写日期: 2026年3月23日
版本: v2.0
适用课程: 国网四川电力AI种子团队培训 – Day3

📋 本日学习目标

完成本日学习后，你将能够：

- ✅ 设计多智能体协作架构
- ✅ 实现Task Planner任务规划
- ✅ 掌握Plan–Execute–Judge执行循环
- ✅ 使用循环模块处理批量任务
- ✅ 发布API供前端调用

预计用时: 8课时（含实操）

第一部分：从单Agent到多智能体

1.1 为什么要拆分为多智能体?

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          单Agent vs 多智能体对比                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  Day2: 单Agent（单体架构）                                                 │
│  ┌──────────────────────────────────────────────┐                        │
│  │  主Agent                                      │                        │
│  │  ├── 用户输入                                 │                        │
│  │  ├── 文档解析                                 │                        │
│  │  ├── 信息提取                                 │                        │
│  │  ├── HTTP调用 × 3                             │                        │
│  │  ├── 数据整合                                 │                        │
│  │  └── 报告生成                                 │                        │
│  └──────────────────────────────────────────────┘                        │
│                                                                          │
│  ❌ 问题:                                                                 │
│     • 模块太多，画布拥挤                                                   │
│     • 功能耦合，难以复用                                                   │
│     • 一处修改，全局影响                                                   │
│     • 团队协作困难                                                         │
│                                                                          │
│  ────────────────────────────────────────────────────────────────────    │
│                                                                          │
│  Day3: 多智能体（分布式架构）                                              │
│  ┌──────────────────────────────────────────────┐                        │
│  │  主Agent (Task Planner)                       │                        │
│  │  └── 负责任务分解和调度                        │                        │
│  └──────────────┬───────────────────────────────┘                        │
│                 │                                                        │
│     ┌───────────┼───────────┐                                            │
│     ▼           ▼           ▼                                            │
│  ┌──────┐   ┌──────┐   ┌──────┐                                          │
│  │文档  │   │数据  │   │评分  │  ◀── Worker Agents                       │
│  │解析  │   │收集  │   │计算  │                                          │
│  │Agent │   │Agent │   │Agent │                                          │
│  └──────┘   └──────┘   └──────┘                                          │
│                                                                          │
│  ✅ 优势:                                                                 │
│     • 模块清晰，职责单一                                                   │
│     • 功能复用，易于维护                                                   │
│     • 独立测试，便于协作                                                   │
│     • 灵活组合，支持扩展                                                   │
└──────────────────────────────────────────────────────────────────────────┘
```

1.2 多智能体架构设计

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              投资问效智能体系统架构                              │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│    【第一层】用户交互层                                                          │
│    ┌──────────────────────────────────────────────────────────────────┐      │
│    │  [上传可研报告]  [输入检索号]  [查看历史]  [导出报告]            │      │
│    └──────────────────────────────────────────────────────────────────┘      │
│                                  │                                            │
│                                  ▼                                            │
│    【第二层】协调层 - Task Planner                                              │
│    ┌──────────────────────────────────────────────────────────────────┐      │
│    │                      主智能体 (Task Planner)                      │      │
│    │   ┌──────────────────────────────────────────────────────────┐   │      │
│    │   │ 职责: 任务分解、依赖管理、进度监控、结果汇总              │   │      │
│    │   │                                                          │   │      │
│    │   │ 核心能力:                                                 │   │      │
│    │   │ ● 解析用户意图 → 确定任务类型                             │   │      │
│    │   │ ● 分析依赖关系 → 生成Action List                          │   │      │
│    │   │ ● 监控执行状态 → 处理异常和重试                           │   │      │
│    │   │ ● 整合子Agent结果 → 生成最终输出                          │   │      │
│    │   └──────────────────────────────────────────────────────────┘   │      │
│    │                                                                  │      │
│    │   输入: 用户请求 (文档/检索号)                                    │      │
│    │   输出: Action List + 执行状态 + 最终结果                         │      │
│    └──────────────────────────────────────────────────────────────────┘      │
│                                  │                                            │
│              ┌───────────────────┼───────────────────┐                        │
│              │                   │                   │                        │
│              ▼                   ▼                   ▼                        │
│    【第三层】执行层 - Worker Agents                                             │
│    ┌────────────────┐  ┌────────────────┐  ┌────────────────┐                │
│    │ Worker Agent │  │ Worker Agent │  │ Worker Agent │                │
│    │   文档解析    │  │   数据收集    │  │   指标评分    │                │
│    ├────────────────┤  ├────────────────┤  ├────────────────┤                │
│    │ 职责:         │  │ 职责:         │  │ 职责:         │                │
│    │ 解析可研报    │  │ 调用业务API   │  │ 8项指标评分   │                │
│    │ 提取关键字段 │  │ 整合返回数据 │  │ 计算综合得分 │                │
│    │               │  │               │  │               │                │
│    │ 输入: 文档    │  │ 输入: 检索号 │  │ 输入: 项目    │                │
│    │ 输出: JSON    │  │ 输出: 数据包 │  │ 输出: 评分    │                │
│    └────────────────┘  └────────────────┘  └────────────────┘                │
│              │                   │                   │                        │
│              ▼                   ▼                   ▼                        │
│    【第四层】工具层                                                             │
│    ┌────────────────┐  ┌────────────────┐  ┌────────────────┐                │
│    │   知识库      │  │   HTTP调用    │  │    代码块     │                │
│    │   文档解析    │  │   业务API     │  │   评分计算    │                │
```

```
│     └──────────────┘    └──────────────┘    └──────────────┘                  │
│                                                                                 │
└─────────────────────────────────────────────────────────────────────────────────┘
```

【设计亮点】

- 分层架构，职责清晰
- 主从协作，易于扩展
- 工具复用，避免重复
- 独立测试，便于维护

第二部分：Plan-Execute-Judge执行循环

2.1 为什么需要PEJ循环?

```
┌──────────────────────────────────────────────────────────────────────┐
│                    复杂任务执行中的常见问题                          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  场景1: API调用失败                                                  │
│  ┌──────────┐    ┌──────────┐      ✗ 网络超时                       │
│  │ Step 1   │──→│ Step 2   │──────→ 失败                           │
│  └──────────┘    └──────────┘                                        │
│       │              │                                               │
│       ▼              ▼                                               │
│   成功继续       失败后怎么办?                                       │
│                                                                      │
│  解决方案: 重试3次, 仍失败则跳过或使用默认值                         │
│                                                                      │
│  ────────────────────────────────────────────────────────────────    │
│                                                                      │
│  场景2: 数据依赖未满足                                               │
│  ┌──────────┐    ┌──────────┐    ┌──────────┐                      │
│  │ Step A   │──→│ Step B   │──→│ Step C   │                      │
│  └──────────┘    └──────────┘    └──────────┘                      │
│       │              │              │                                │
│       ▼              ▼              ▼                                │
│    提取编号      查询数据       C依赖A和B                            │
│                   失败                                               │
│                                                                      │
│  解决方案: 检查依赖状态, 未满足时等待或报错                          │
│                                                                      │
│  ────────────────────────────────────────────────────────────────    │
│                                                                      │
│  场景3: 结果质量不达标                                               │
│  ┌──────────┐    ┌──────────┐                                       │
│  │ 生成报告 │──→│ 质量检查 │──→ 评分低于60分                       │
│  └──────────┘    └──────────┘                                        │
│                       │                                              │
│                       ▼                                              │
│                  需要补充数据                                        │
│                                                                      │
│  解决方案: 返回补充数据步骤, 重新生成                                │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

2.2 PEJ循环详解

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    Plan-Execute-Judge 执行循环                           │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ╔══════════════════════════════════════════════════════════════════════╗
│
│   ╟  Phase 1: PLAN（规划阶段）                                          ╟ │
│   ╠══════════════════════════════════════════════════════════════════════╣
│
│   ╟                                                                      ╟
│
│   ╟  【输入】用户请求（文档/检索号）                                     ╟ │
│   ╟                                                                      ╟
│
│   ╟  【处理】Task Planner                                                ╟ │
│   ╟  1. 解析用户意图 → 确定任务类型                                      ╟ │
│   ╟  2. 分析可用资源 → 确定可调用的Agent/工具                            ╟ │
│   ╟  3. 梳理依赖关系 → 确定执行顺序                                      ╟ │
│   ╟  4. 生成Action List → 结构化执行计划                                 ╟ │
│   ╟                                                                      ╟
│
│   ╟  【输出】Action List（JSON数组）                                     ╟ │
│   ╟  [                                                                   ╟
│
│   ╟    {step:1, flag:"REPORT_ANALYSIS", depends_on:[]},                  ╟ │
│   ╟    {step:2, flag:"GET_PROJECT_BASIC", depends_on:[1]},               ╟ │
│   ╟    {step:3, flag:"SCORE_CALCULATION", depends_on:[2]},               ╟ │
│   ╟    ...                                                               ╟ │
│   ╟  ]                                                                   ╟
│
│   ╟                                                                      ╟
│
│   ╚══════════════════════════════════════════════════════════════════════╝
│
│                                  │                                       │
│                                  ▼                                       │
│   ╔══════════════════════════════════════════════════════════════════════╗
│
│   ╟  Phase 2: EXECUTE（执行阶段）                                       ╟ │
│   ╠══════════════════════════════════════════════════════════════════════╣
│
│   ╟                                                                      ╟
│
│   ╟  【输入】当前Action（从Action List中取出）                           ╟ │
│   ╟                                                                      ╟
│
│   ╟  【检查】依赖是否满足?                                               ╟ │
│   ╟  ├─→ 未满足 → 等待或报错                                             ╟ │
```

```
│    │    └─→ 已满足 → 继续执行                                          │ │
│    │                                                                    │
│
│    │   【路由】根据flag调用对应Agent/工具                                 │ │
│    │   flag:"REPORT_ANALYSIS" → 调用文档解析Agent                        │ │
│    │   flag:"GET_PROJECT_BASIC" → 调用HTTP模块                           │ │
│    │   flag:"SCORE_CALCULATION" → 调用评分Agent                          │ │
│    │                                                                    │
│
│    │   【输出】执行结果（success + data 或 error）                        │ │
│    │                                                                    │
│
│    └────────────────────────────────────────────────────────────────────┘
│
│                                  │                                        │
│                                  ▼                                        │
│    ╔═══════════════════════════════════════════════════════════════════╗
│
│    ╠  Phase 3: JUDGE（校验阶段）                                        ╠ │
│    ╚═══════════════════════════════════════════════════════════════════╝
│
│    │                                                                    │
│
│    │   【检查1】执行是否成功?                                             │ │
│    │   ├─→ 失败 → 进入错误处理流程                                       │ │
│    │   └─→ 成功 → 继续检查2                                              │ │
│    │                                                                    │
│
│    │   【检查2】结果是否完整?                                             │ │
│    │   ├─→ 不完整 → 记录缺失字段，标记为部分成功                          │ │
│    │   └─→ 完整 → 继续检查3                                              │ │
│    │                                                                    │
│
│    │   【检查3】质量是否达标?                                             │ │
│    │   ├─→ 不达标 → 返回PLAN阶段，补充数据                                │ │
│    │   └─→ 达标 → 执行成功                                               │ │
│    │                                                                    │
│
│    └────────────────────────────────────────────────────────────────────┘
│
│                                  │                                        │
│                       ┌──────────┼──────────┐                            │
│                       │          │          │                            │
│                       ▼          ▼          ▼                            │
│                  ┌─────────┐  ┌─────────┐  ┌─────────┐                  │
│                  │ 成功    │  │ 重试    │  │ 失败    │                  │
│                  │         │  │         │  │         │                  │
│                  │ 存储结果│  │ 计数+1  │  │ 记录日志│                  │
│                  │ 检查Next│  │ <3次?   │  │ 跳过    │                  │
│                  │         │  │ 是→重试 │  │ 继续    │                  │
│                  │ Yes→Next│  │ 否→失败 │  │         │                  │
```

```
│                  │ No→结束 │    │         │     │         │                   │
│                  └─────────┘    └─────────┘     └─────────┘                   │
│                                                                                │
└────────────────────────────────────────────────────────────────────────────────┘

【循环控制逻辑】

成功路径: Judge成功 → 存储结果 → 检查是否还有Next Action
                                  ↓
                          Yes → 回到EXECUTE执行下一个
                                  ↓
                          No  → 进入Phase 4: GENERATE生成报告

失败路径: Judge失败 → 重试计数+1 → 检查重试次数<3?
                                  ↓
                          Yes → 回到EXECUTE重试
                                  ↓
                          No  → 记录错误 → 标记为失败 → 继续下一个
```

2.3 在灵搭中实现PEJ循环

方案A: 使用记忆变量实现循环（推荐）

```
【核心思路】
使用代码块存储执行状态，通过"智能对话"模块进行循环控制

流程设计:

[开始]
  |
  ▼
[代码块: 初始化状态]
  |
  ├─→ current_step: 0
  ├─→ total_steps: 7
  ├─→ results: {}        # 存储每步结果
  ├─→ retry_count: {}    # 每步重试次数
  └─→ status: 'running'
  |
  ▼
[智能对话: Task Planner生成Plan]
  |
  ├─→ 输入: user_request
  ├─→ 输出: action_list
  |
  ▼
[代码块: 存储Plan到记忆变量]
  |
  ▼
╔═══════════════════════════════════════════════════════════╗
║  【循环开始】                                              ║
║                                                           ║
║  [智能对话: 执行当前Step]                                  ║
║    |                                                      ║
║    ├─→ 从memory读取current_step                            ║
║    ├─→ 获取对应Action                                      ║
║    ├─→ 调用对应Agent/工具                                  ║
║    └─→ 执行结果存入result                                  ║
║    |                                                      ║
║    ▼                                                      ║
║  [代码块: Judge校验]                                        ║
║    |                                                      ║
║    ├─→ isValid: 检查必需字段                               ║
║    ├─→ canRetry: 重试次数<3?                               ║
║    └─→ 更新memory中的retry_count                           ║
║    |                                                      ║
║    ├─→ 失败且可重试 → 回到【智能对话】重试                   ║
║    ├─→ 失败且不可重试 → 记录错误，current_step++            ║
```

```
│     └─→ 成功 → current_step++, 存储结果                    │
│     |                                                      │
│     ▼                                                      │
│   [代码块: 检查是否完成]                                    │
│     |                                                      │
│     ├─→ current_step < total_steps → 回到【循环开始】       │
│     └─→ current_step >= total_steps → 退出循环              │
│                                                            │
└────────────────────────────────────────────────────────────┘
  |
  ▼
[智能对话: 整合结果生成报告]
  |
  ▼
[结束]
```

代码块配置示例:

```
# 代码块: 初始化状态
节点名称: 初始化执行状态
语言: JavaScript

代码: |
  /**
   * 【功能】初始化PEJ循环的状态变量
   * 【说明】这些变量将在整个执行过程中被更新和传递
   */
  function userFunction(param) {
      return {
          // 【核心】当前执行步骤
          current_step: 0,

          // 【配置】总步骤数（从Plan获取）
          total_steps: 0,

          // 【存储】每步的执行结果
          results: {},

          // 【控制】每步的重试次数
          retry_count: {},

          // 【元数据】Action List
          action_list: [],

          // 【状态】整体执行状态
          status: 'initialized',  // initialized/running/completed/failed

          // 【日志】执行日志
          logs: []
      };
  }

出参:
  - initial_state (字符串)  # JSON格式，后续模块通过{{initial_state}}引用

---

# 代码块: Judge校验
节点名称: 校验执行结果
语言: JavaScript

入参:
  - current_state: 上游传递的状态
  - step_result: 当前步骤执行结果
  - step_index: 当前步骤索引

代码: |
  function userFunction(param) {
      // 【步骤1】解析输入
```

```
var state = JSON.parse(param['current_state']);
var result = JSON.parse(param['step_result']);
var stepIdx = parseInt(param['step_index']);

// 【步骤2】获取当前Action的配置
var action = state.action_list[stepIdx];
var requiredFields = action.expected_output || [];

// 【步骤3】检查必需字段
var missing = [];
for (var i = 0; i < requiredFields.length; i++) {
    var field = requiredFields[i];
    if (!result[field] && result[field] !== 0) {
        missing.push(field);
    }
}

// 【步骤4】检查执行状态
var isSuccess = result.success !== false;
var isComplete = missing.length === 0;

// 【步骤5】重试逻辑
var retryKey = 'step_' + stepIdx;
state.retry_count[retryKey] = (state.retry_count[retryKey] || 0) + 1;
var retryCount = state.retry_count[retryKey];
var canRetry = retryCount < 3 && !isSuccess;

// 【步骤6】更新状态
if (isSuccess && isComplete) {
    // 成功：存储结果，进入下一步
    state.results[retryKey] = result;
    state.current_step = stepIdx + 1;
    state.logs.push({
        step: stepIdx,
        status: 'success',
        timestamp: new Date().toISOString()
    });
} else if (canRetry) {
    // 可重试：保持当前步骤，增加重试计数
    state.logs.push({
        step: stepIdx,
        status: 'retry',
        retry_count: retryCount,
        error: result.error || '字段缺失: ' + missing.join(',')
    });
} else {
    // 失败：记录错误，进入下一步（跳过）
    state.results[retryKey] = {
        success: false,
        error: result.error || '字段缺失: ' + missing.join(','),
        partial_data: result
```

```
            };
            state.current_step = stepIdx + 1;
            state.logs.push({
                step: stepIdx,
                status: 'failed',
                error: result.error || '超过最大重试次数'
            });
        }

        // 【步骤7】检查是否完成
        var isFinished = state.current_step >= state.total_steps;

        return {
            updated_state: JSON.stringify(state),
            is_valid: isSuccess && isComplete,
            can_retry: canRetry,
            is_finished: isFinished,
            progress: state.current_step + '/' + state.total_steps,
            message: isSuccess && isComplete ? '步骤执行成功' :
                     canRetry ? '准备重试（第' + retryCount + '次）' :
                     '步骤执行失败，已跳过'
        };
    }

出参:
  - updated_state（字符串）  # 更新后的状态
  - is_valid（布尔型）       # 是否有效
  - can_retry（布尔型）      # 是否可以重试
  - is_finished（布尔型）    # 是否全部完成
  - progress（字符串）       # 进度显示
```

第三部分：子智能体设计详解

3.1 Task Planner Agent（任务规划器）

配置详解:

```
Agent名称: Task Planner_投资问效

系统提示词: |
  【角色】你是投资问效系统的任务规划助手。

  【核心职责】
  1. 解析用户输入，理解用户意图
  2. 分析任务依赖关系
  3. 生成结构化的Action List

  【输入类型判断】
  根据用户输入判断任务类型:
  - Type A: 用户上传了可研报告文档
    → 需要: 文档解析 → 数据收集 → 评分计算 → 报告生成
  - Type B: 用户提供了项目检索号
    → 需要: 数据收集 → 评分计算 → 报告生成
  - Type C: 用户询问系统功能/评分标准
    → 需要: FAQ回答（无需调用Agent）

  【可用工具清单】

  | Flag | 工具名称 | 功能说明 | 必需输入 | 输出字段 |
  |------|---------|---------|---------|---------|
  | REPORT_ANALYSIS | 文档解析 | 解析可研报告，提取关键字段 | report_text |
retrieval_no, project_name, forecast_load |
  | GET_PROJECT_BASIC | 基本信息查询 | 获取项目基础属性 | retrieval_no |
voltage_level, project_type, dates |
  | GET_PROJECT_INVESTMENT | 投资数据查询 | 获取投资金额 | retrieval_no |
estimated_cost, actual_cost |
  | GET_PROJECT_OPERATION | 运行指标查询 | 获取运行数据 | retrieval_no |
load_data, n1_rate, device_load |
  | PROJECT_TYPE_EXTRACTION | 项目类型提取 | 识别项目分类 | project_data |
project_type |
  | SCORE_CALCULATION | 指标评分 | 计算8项指标得分 | all_data | scores,
total_score |
  | GENERATE_REPORT | 报告生成 | 生成分析报告 | all_data, scores |
report_markdown |

  【依赖关系规则】
  - REPORT_ANALYSIS 无依赖（可立即执行）
  - GET_PROJECT_BASIC/INVESTMENT/OPERATION 依赖 REPORT_ANALYSIS 的
retrieval_no
  - SCORE_CALCULATION 依赖所有数据收集步骤完成
  - GENERATE_REPORT 依赖 SCORE_CALCULATION

  【输出格式 - 严格遵循】
  必须输出JSON格式的Action List，不要其他内容:

  {
    "plan_id": "plan_{{timestamp}}",
    "task_type": "A|B|C",
```

```
    "total_steps": 7,
    "actions": [
      {
        "step": 1,
        "flag": "REPORT_ANALYSIS",
        "description": "解析可研报告，提取检索号等关键信息",
        "depends_on": [],
        "input": {
          "report_text": "{{user_input}}"
        },
        "expected_output": ["retrieval_no", "project_name",
"forecast_load"]
      },
      {
        "step": 2,
        "flag": "GET_PROJECT_BASIC",
        "description": "获取项目基本信息",
        "depends_on": [1],
        "input": {
          "retrieval_no": "{{step1.retrieval_no}}"
        },
        "expected_output": ["voltage_level", "project_type", "location"]
      }
      // ... 更多步骤
    ]
  }

  【约束条件】
  • step必须从1开始，连续递增
  • depends_on必须引用已定义的step
  • flag必须从工具清单中选择
  • expected_output必须明确列出所有必需字段

模型: 光明电力大模型V2
回复字数上限: 4000
温度: 0.1  # 低温度保证输出稳定
```

3.2 子Agent配置规范

子Agent通用配置模板:

```
# ═══════════════════════════════════════════════════════════════════
# 【子Agent配置模板】
# ═══════════════════════════════════════════════════════════════════

Agent名称: 【功能】Agent_【创建者姓名】

# 【必需】Agent对话结束模块配置
Agent对话结束模块:
  展示确认弹窗: 是  # 建议开启，便于调试

  子Agent运行结束后确认: |
    【功能】已完成，是否继续?

    {{result_summary}}  # 显示执行结果摘要

  重新执行母Agent: "重新执行"  # 用户不满意时
  满意，继续执行母Agent: "继续"  # 用户满意时
  不满意，重新执行子Agent: "重试"  # 重试当前子Agent

# 【建议】输入输出规范
输入处理:
  - 使用"用户提问"模块接收主Agent传递的参数
  - 在系统提示词中说明期望的输入格式
  - 验证输入完整性，缺失时友好提示

输出处理:
  - 使用JSON格式输出结构化数据
  - 包含success字段标识执行状态
  - 失败时提供详细的error信息
```

3.3 数据收集Agent完整示例

```
Agent名称: 数据收集Agent_投资问效

【流程设计】
[用户提问]  ◀── 接收主Agent传递的retrieval_no
     |
     ▼
[确定回复]  ◀── 提示"正在查询项目数据..."
     |
     ▼
[并行HTTP调用]
     ├─→ [HTTP: 基本信息] ──┐
     ├─→ [HTTP: 投资数据] ──┼── 任一激活
     └─→ [HTTP: 运行指标] ──┘
     |
     ▼
[代码块: 数据整合]
     |
     ▼
[智能对话: 结果格式化]
     |
     ▼
[Agent对话结束]  ◀── 返回主Agent

【代码块: 数据整合】
语言: JavaScript

入参:
  - basic_response: {{HTTP基本信息.output}}
  - investment_response: {{HTTP投资数据.output}}
  - operation_response: {{HTTP运行指标.output}}
  - retrieval_no: {{用户提问.input}}

代码: |
  function userFunction(param) {
      try {
          // 【解析】3个API的响应
          var basic = JSON.parse(param['basic_response']);
          var investment = JSON.parse(param['investment_response']);
          var operation = JSON.parse(param['operation_response']);
          var retrievalNo = param['retrieval_no'];

          // 【校验】检查所有API是否成功
          var errors = [];
          if (basic.code !== 200) errors.push('基本信息查询失败: ' +
basic.message);
          if (investment.code !== 200) errors.push('投资数据查询失败: ' +
investment.message);
          if (operation.code !== 200) errors.push('运行指标查询失败: ' +
```

```
operation.message);

          if (errors.length > 0) {
              return {
                  success: false,
                  error: errors.join('; '),
                  retrieval_no: retrievalNo,
                  partial_data: {
                      basic: basic.data,
                      investment: investment.data,
                      operation: operation.data
                  }
              };
          }

          //  【整合】构建统一数据对象
          var result = {
              success: true,
              retrieval_no: retrievalNo,
              query_time: new Date().toISOString(),
              data: {
                  basic: basic.data,
                  investment: investment.data,
                  operation: operation.data
              },
              summary: {
                  project_name: basic.data['项目名称'],
                  voltage_level: basic.data['电压等级'],
                  has_all_data: !!(basic.data && investment.data &&
operation.data)
              }
          };

          return result;

      } catch(e) {
          return {
              success: false,
              error: '数据整合失败: ' + e.message,
              retrieval_no: param['retrieval_no']
          };
      }
  }

出参:
  - success (布尔型)
  - retrieval_no (字符串)
  - data (对象)
  - summary (对象)
  - error (字符串)
```

第四部分：循环模块高级应用

4.1 批量项目评估

```
【场景】年度投资问效评估
● 项目数量：50-100个
● 输入：Excel文件包含项目列表
● 输出：批量评估报告

【流程设计】

[用户上传Excel]
      |
      ▼
[代码块：解析Excel]
      |
      ▼
[提取项目列表]  ◀── 数组格式 [{retrieval_no, project_name}, ...]
      |
      ▼
╔═══════════════════════════════════════════════════════════╗
║  【循环模块】遍历项目列表                                  ║
║                                                           ║
║  信息输入：project_list（数组）                            ║
║                                                           ║
║  循环单元内部：                                            ║
║    1. [信息提取] 获取当前项目 {{loop.item.retrieval_no}}  ║
║    2. [子Agent调用] 数据收集Agent                          ║
║    3. [子Agent调用] 评分计算Agent                          ║
║    4. [代码块] 存储评估结果到数组                          ║
║    5. [确定回复] 显示进度 {{loop.index+1}}/{{loop.length}} ║
║                                                           ║
║  循环变量：                                                ║
║    ● {{batch_loop.item}} - 当前项目数据                    ║
║    ● {{batch_loop.index}} - 当前索引（从0开始）            ║
║    ● {{batch_loop.length}} - 总数                          ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
      |
      ▼
[代码块：汇总所有结果]
      |
      ▼
[智能对话：生成批量评估报告]
      |
      ▼
[提供下载链接]
```

进度提示实现:

```
代码块: 更新进度
语言: JavaScript

入参:
  - current_index: {{batch_loop.index}}
  - total_count: {{batch_loop.length}}
  - current_result: {{评分计算Agent.output}}
  - accumulated_results: 上游传递的累积结果

代码: |
  function userFunction(param) {
      var current = parseInt(param['current_index']) + 1;
      var total = parseInt(param['total_count']);
      var currentResult = JSON.parse(param['current_result']);

      // 【累积】将当前结果加入列表
      var results = JSON.parse(param['accumulated_results'] || '[]');
      results.push({
          index: current,
          retrieval_no: currentResult.retrieval_no,
          project_name: currentResult.project_name,
          total_score: currentResult.total_score,
          grade: currentResult.grade,
          success: currentResult.success
      });

      // 【计算】进度百分比
      var progress = (current / total * 100).toFixed(1);
      var remaining = total - current;
      var eta = remaining * 3; // 假设每个项目3秒

      return {
          progress_message: `已完成 ${current}/${total} 个项目
(${progress}%)`,
          eta_message: `预计剩余时间: ${eta}秒`,
          all_results: JSON.stringify(results),
          is_complete: current >= total,

          // 【统计】成功/失败数量
          success_count: results.filter(r => r.success).length,
          failed_count: results.filter(r => !r.success).length
      };
  }
```

4.2 嵌套循环（复杂依赖）

```
【双层循环架构】

外层循环：遍历项目列表
  |
  ▼
内层循环：遍历Action列表
      |
      ├─→ 检查依赖是否满足
      ├─→ 执行Action
      └─→ 校验结果

【实现方式】
使用记忆变量传递状态，通过条件分支控制循环

[外层循环开始]
      |
      ▼
[代码块：为当前项目生成Action List]
      |
      ▼
╔═══════════════════════════════════════════════════════════════╗
║ 内层循环（使用记忆变量实现）                                  ║
║                                                               ║
║  [代码块：检查当前Action]                                     ║
║     |                                                         ║
║     ├─→ 依赖满足？                                            ║
║     |    ├─→ 否 → 等待/跳过                                   ║
║     |    └─→ 是 → 继续                                        ║
║     |                                                         ║
║     ▼                                                         ║
║  [执行当前Action]                                             ║
║     |                                                         ║
║     ▼                                                         ║
║  [代码块：校验结果]                                           ║
║     |                                                         ║
║     ├─→ 成功 → 存储结果，action_index++                       ║
║     ├─→ 失败且可重试 → 重试计数++                             ║
║     └─→ 失败且不可重试 → 记录错误，action_index++             ║
║     |                                                         ║
║     ▼                                                         ║
║  [代码块：检查内层循环是否完成]                                ║
║     |                                                         ║
║     ├─→ action_index < action_count → 回到【内层循环开始】     ║
║     └─→ action_index >= action_count → 退出内层循环           ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝

      |
      ▼
```

```
[代码块: 项目评估完成, project_index++]
      |
      ▼
[检查外层循环是否完成]
```

第五部分：API发布与集成

5.1 发布流程

发布步骤:

```
步骤1: 测试验证
  - 在试运行模式下测试完整流程
  - 验证各种输入场景
  - 确认错误处理正常

步骤2: 点击发布
  - 点击右上角"···"按钮
  - 选择"发布"选项
  - 进入发布页面

步骤3: 创建API链接
  - 选择"API服务"标签
  - 点击"创建链接"
  - 系统自动生成API信息

步骤4: 记录API信息
  - Uuid: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
  - AuthKey: your-auth-key
  - AuthSecret: your-auth-secret

步骤5: 权限设置（可选）
  - 设置IP白名单
  - 配置调用频率限制
  - 设置有效期
```

5.2 前端调用示例

```
// ═══════════════════════════════════════════════════════════════
// 【API配置】
// ═══════════════════════════════════════════════════════════════
const API_CONFIG = {
  baseUrl: import.meta.env.VITE_API_BASE_URL ||
'https://api.xxx.com/agent',
  uuid: import.meta.env.VITE_AGENT_UUID || 'your-uuid-here',
  authKey: import.meta.env.VITE_AUTH_KEY || 'your-auth-key'
};

// ═══════════════════════════════════════════════════════════════
// 【核心函数】调用投资问效Agent
// ═══════════════════════════════════════════════════════════════
async function callInvestmentAgent(query, context = {}) {
  try {
    const response = await fetch(
      `${API_CONFIG.baseUrl}/${API_CONFIG.uuid}`,
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${API_CONFIG.authKey}`
        },
        body: JSON.stringify({
          query,           // 【输入】用户查询（文档文本或检索号）
          context: {
            ...context,
            timestamp: new Date().toISOString(),
            client: 'web'
          }
        })
      }
    );

    // 【错误处理】HTTP层面错误
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }

    const result = await response.json();

    // 【错误处理】Agent层面错误
    if (result.code !== 200) {
      throw new Error(result.message || 'Agent返回错误');
    }

    // 【解析】提取并解析报告
    return parseAgentResponse(result.data.answer);
```

```
  } catch (error) {
    console.error('API调用失败:', error);
    throw error;
  }
}

// ═══════════════════════════════════════════════════════════════════
//  【解析函数】解析Agent返回的报告
// ═══════════════════════════════════════════════════════════════════
function parseAgentResponse(answer) {
  try {
    //  【方式1】从Markdown代码块中提取JSON
    const jsonMatch = answer.match(/```json\n([\s\S]*?)\n```/);
    if (jsonMatch) {
      const data = JSON.parse(jsonMatch[1]);
      return {
        type: 'structured',
        data: data,
        report: answer  // 原始Markdown报告
      };
    }

    //  【方式2】尝试直接解析整个回答
    const directParse = JSON.parse(answer);
    return {
      type: 'json',
      data: directParse,
      report: answer
    };

  } catch (e) {
    //  【回退】返回原始文本
    return {
      type: 'text',
      data: null,
      report: answer,
      parseError: e.message
    };
  }
}

// ═══════════════════════════════════════════════════════════════════
//  【使用示例】
// ═══════════════════════════════════════════════════════════════════

// 示例1: 通过检索号查询
async function queryByRetrievalNo(retrievalNo) {
  const result = await callInvestmentAgent(retrievalNo, {
    action: 'query_by_retrieval_no',
    retrievalNo: retrievalNo
```

```
  });

  console.log('项目数据:', result.data);
  console.log('评估报告:', result.report);
  return result;
}

// 示例2: 上传文档分析
async function analyzeReport(reportText) {
  const result = await callInvestmentAgent(reportText, {
    action: 'analyze_report',
    source: 'document_upload'
  });

  return result;
}

// 使用
queryByRetrievalNo('798-K131538-A01')
  .then(result => {
    // 在前端展示结果
    displayReport(result);
  })
  .catch(error => {
    // 显示错误提示
    showError(error.message);
  });
```

第六部分：测试与优化

6.1 测试清单

测试项	测试方法	通过标准
Task Planner	输入不同类型请求	生成正确的Action List
子Agent调用	单独测试每个子Agent	返回正确格式数据
PEJ循环	模拟失败场景	正确重试或跳过
批量处理	上传包含5个项目的Excel	正确生成批量报告
API集成	使用curl/postman调用	返回200和正确数据
并发测试	同时发起10个请求	全部正确处理

6.2 性能优化建议

```
Agent端优化:
  - 缓存策略: 相同检索号结果缓存24小时
  - 并发控制: 限制同时运行的子Agent数量（建议<5）
  - 超时处理: API调用设置30秒超时
  - 降级策略: API失败时返回部分结果

调用端优化:
  - 连接池: 复用HTTP连接
  - 异步调用: 非阻塞调用
  - 结果缓存: 前端缓存最近10次查询
```

附录：参考文件

📎 附件1：API接口文档

文件: `API使用说明.md`

📎 附件2：案例流程说明

文件: `案例智能体流程说明.pdf`

📎 附件3：业务汇报材料

文件: `35千伏及以上已投产项目投资问效智能场景汇报-0623v5.1.pdf`

📎 附件4：架构示意图

文件: `案例智能体示意图.pdf`

文档信息

- 编写日期: 2026年3月23日
- 版本: v2.0
- 编写人: AI教学团队
- 适用对象: 国网四川电力AI种子团队